Design an AI-Powered Capacity Planning & Decision Support Platform

Problem Statement

Design a platform that collects infrastructure and business metrics from a large-scale cloud environment (10,000+ services, millions of VMs/containers), forecasts future resource demand using ML models, optimizes capacity allocation to minimize cost while meeting SLAs, and provides AI-assisted decision support — including natural language queries, automated reports, and what-if simulations. The platform serves infrastructure teams, finance (cost forecasting), and leadership (strategic capacity planning).

Clarifying Questions to Ask

1. **Scale**: How many services / resources are we tracking? (10K+ services, 1M+ VMs/containers)
2. **Metrics volume**: How many time-series metrics? (Millions of distinct metric streams)
3. **Forecast horizon**: Short-term (hours/days) for auto-scaling, or long-term (quarters/years) for procurement?
4. **Decision scope**: Just compute/storage, or also network, licenses, people?
5. **Automation level**: Read-only recommendations, or can the system auto-execute actions (auto-scale, auto-provision)?
6. **Users**: Infrastructure engineers, SREs, finance, leadership — different views/permissions?
7. **LLM integration**: Natural language queries only, or also automated report generation and what-if simulations?
8. **Multi-cloud**: Single cloud (Azure), or multi-cloud (Azure + AWS + GCP)?
9. **Cost optimization**: Include RI/savings plan recommendations? Spot instance optimization?
10. **Compliance**: Any data residency or audit requirements for the platform itself?

Requirements & Sizing

Functional

- **Metrics Ingestion**: Collect compute, storage, network, and cost metrics from cloud providers, APM tools, and business systems

- **Forecasting**: Generate demand forecasts at multiple horizons (hours, days, weeks, quarters) with confidence intervals
- **Optimization**: Recommend optimal resource allocation (VM SKUs, reserved instances, scaling policies) that minimizes cost while meeting SLAs
- **Decision Support Dashboard**: Interactive UI showing current usage, forecasts, recommendations, and what-if scenarios
- **Natural Language Interface**: Ask questions in English ("Will we need more GPUs next quarter?") and get data-grounded answers
- **Automated Reports**: Weekly/monthly capacity reports auto-generated and sent to stakeholders
- **What-If Simulations**: "What if traffic grows 3x?" / "What if we lose a region?" → model the impact
- **Alerting**: Proactive alerts when forecasts predict capacity risks (saturation, budget overrun)
- **Model Lifecycle**: Train, validate, deploy, monitor, and retrain forecasting models automatically
- **Audit Trail**: Log all recommendations, human decisions, and automated actions

Non-functional

Metric	Target
Metrics ingestion	**5M+ metric data points/second**
Metrics storage	**2+ years** of historical data at variable resolution
Forecast latency	< **30 seconds** for on-demand single-service forecast
Batch forecast	< **2 hours** for full fleet (10K+ services)
Dashboard latency	< **2 seconds** for any dashboard query
NL query latency	< **10 seconds** for natural language questions
Alert latency	< **5 minutes** from metric to alert
Availability	**99.9%** for dashboards, **99.99%** for ingestion pipeline
Data freshness	< **5 minutes** from source metric to platform

Capacity Estimates

```
Metrics:
- 10K services × 500 metrics/service = 5M distinct time-series
- Each metric: data point every 60 seconds = 5M points/minute
- Data point: ~50 bytes (timestamp, value, tags)
- Ingestion: 5M × 50B / 60s ≈ 4 MB/second (raw)
```

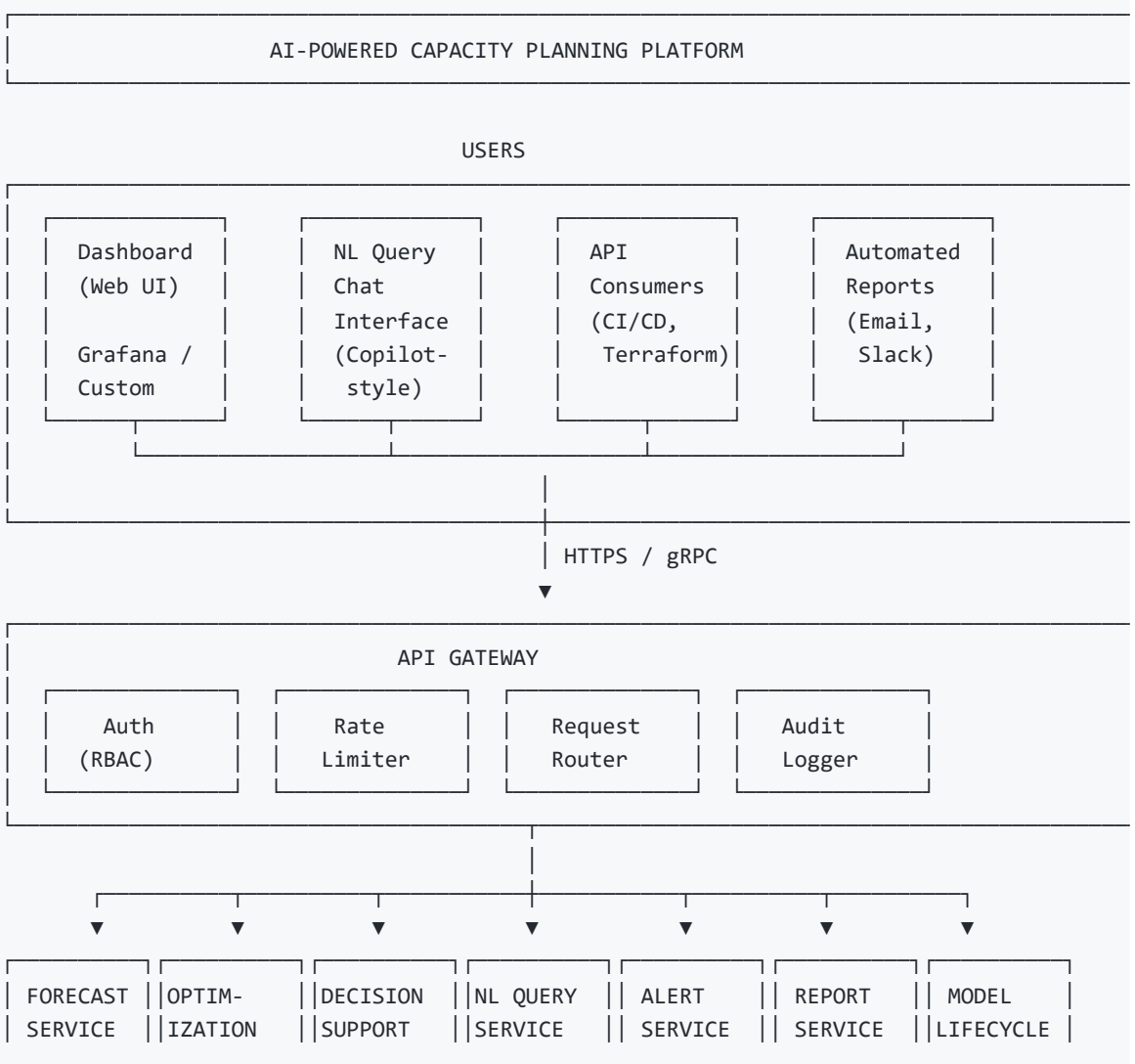
```
- With tags/metadata: ~40 MB/second ingested
- Daily storage: 5M series × 1440 points × 50B ≈ 360 GB/day (raw)
- 2 years retention: ~250 TB (with downsampling: ~25 TB)

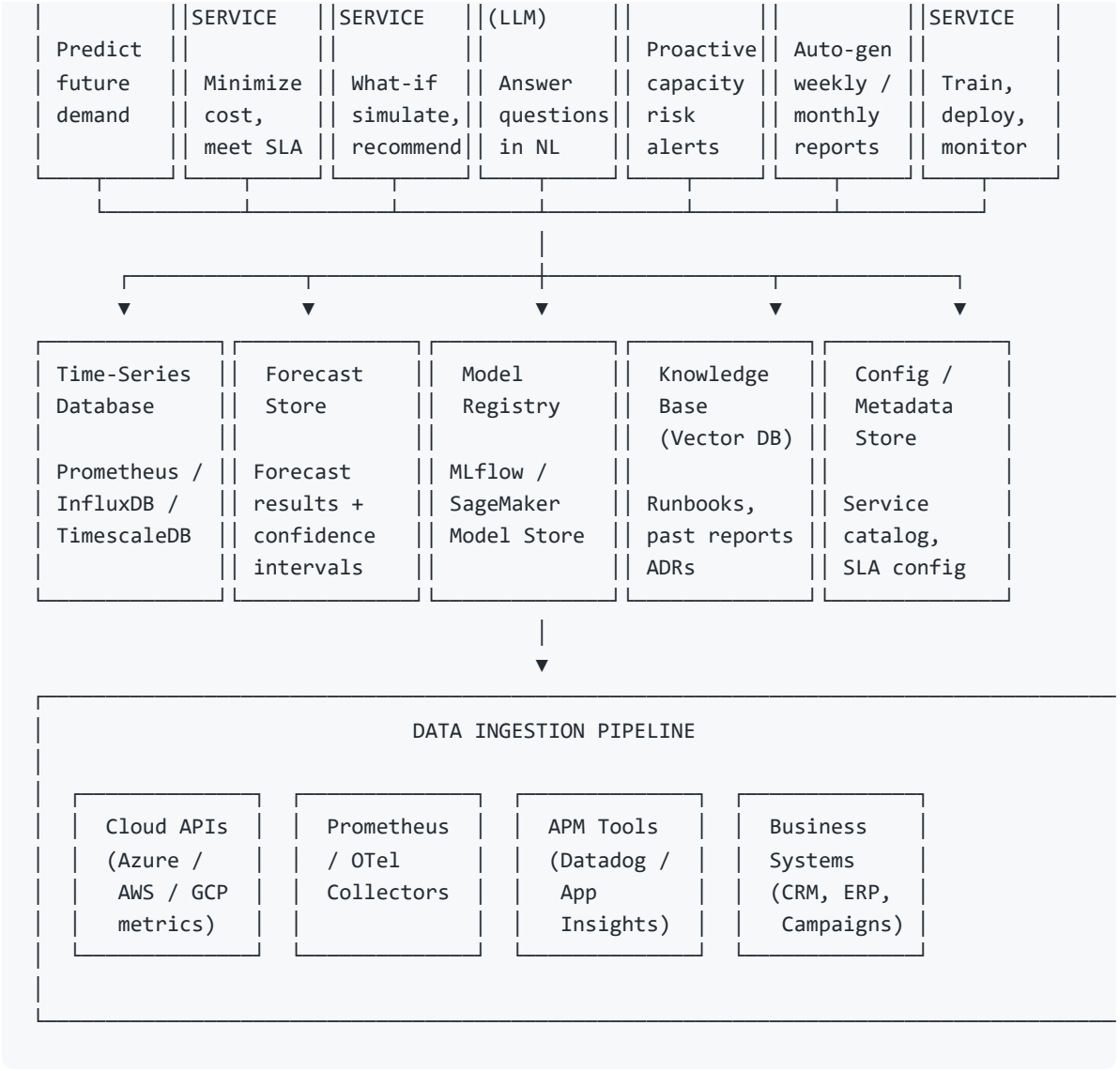
Forecasts:
- 5M series × 4 horizons × daily reforecast = 20M forecasts/day
- Forecast result: ~500 bytes (point + confidence interval + metadata)
- Forecast storage: 20M × 500B ≈ 10 GB/day

Queries:
- 500 engineers × 20 dashboard views/day = 10K dashboard queries/day
- 50 NL queries/day (growing)
- Batch reports: 100 automated reports/week

Optimization:
- Full fleet optimization: Run nightly (1 run = score 10K services)
- What-if simulations: 10-50/day (on-demand)
```

High-Level Architecture





API Design

Forecast APIs

Method	Endpoint	Description
`GET`	`/v1/forecasts/{service_id}?horizon={days}&metric={cpu|mem|storage}`	Get demand forecast for a service
`POST`	`/v1/forecasts/batch`	Trigger batch forecast for all services
`GET`	`/v1/forecasts/{service_id}/history`	Get past forecasts + actual values (accuracy tracking)

Optimization APIs

Method	Endpoint	Description
`POST`	`/v1/optimize`	Run optimization for a scope (service, team, org)
`GET`	`/v1/recommendations/{service_id}`	Get current recommendations for a service
`POST`	`/v1/recommendations/{id}/approve`	Approve a recommendation (triggers action)
`POST`	`/v1/recommendations/{id}/reject`	Reject a recommendation (with reason)

What-If & Decision Support APIs

Method	Endpoint	Description
`POST`	`/v1/simulate`	Run what-if simulation
`GET`	`/v1/capacity/{service_id}/status`	Current capacity status + risk level
`POST`	`/v1/query`	Natural language query (LLM-powered)

Report APIs

Method	Endpoint	Description
`POST`	`/v1/reports/generate`	Generate a capacity report on demand
`GET`	`/v1/reports/{id}`	Retrieve a generated report
`GET`	`/v1/reports?schedule=weekly`	List scheduled reports

Forecast Request / Response

```
GET /v1/forecasts/svc-12345?horizon=30&metric=cpu&confidence=0.95

{
  "service_id": "svc-12345",
  "service_name": "payment-api",
  "metric": "cpu_utilization_percent",
  "forecast_horizon_days": 30,
  "generated_at": "2026-03-03T21:00:00Z",
  "model_used": "ensemble_v3 (Prophet + XGBoost)",
```

```
  "model_accuracy_mape": 7.2,
  "current_value": 62.5,
  "forecast": [
    {
      "date": "2026-03-04",
      "predicted": 63.1,
      "lower_95": 58.2,
      "upper_95": 68.0
    },
    {
      "date": "2026-03-05",
      "predicted": 63.8,
      "lower_95": 57.5,
      "upper_95": 70.1
    }
  ],
  "alerts": [
    {
      "type": "capacity_risk",
      "message": "CPU forecast to exceed 80% threshold in 18 days",
      "severity": "warning",
      "confidence": 0.89
    }
  ],
  "recommendation": {
    "id": "rec-abc123",
    "action": "scale_up",
    "detail": "Add 15 D8s_v5 VMs to payment-api-eastus cluster",
    "estimated_cost_increase": "$4,200/month",
    "deadline": "2026-03-18",
    "status": "pending_approval"
  }
}
```

Natural Language Query Request / Response

```
POST /v1/query
{
  "question": "Will we need more GPU capacity for the ML training cluster next quart
  "context": {
    "team": "ml-platform",
    "scope": "gpu-clusters"
  }
}

{
  "answer": "Based on the current forecast, your ML training cluster GPU utilization
  "sources": [
    { "type": "forecast", "id": "forecast-gpu-cluster-q2", "confidence": 0.87 },
    { "type": "document", "title": "ML Platform Q2 Roadmap", "section": "Training In
    { "type": "metric", "name": "gpu_utilization_percent", "current": 61.2 }
  ],
```

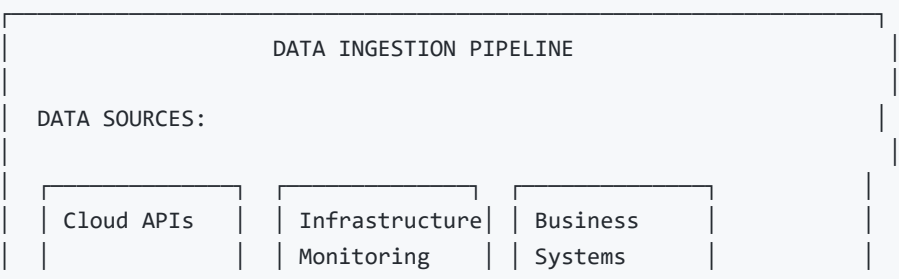
```
  "followup_suggestions": [
    "Show me the GPU utilization forecast chart",
    "What's the cost comparison between on-demand and spot for training?",
    "What if we delay the LLM fine-tuning to May?"
  ]
}
```

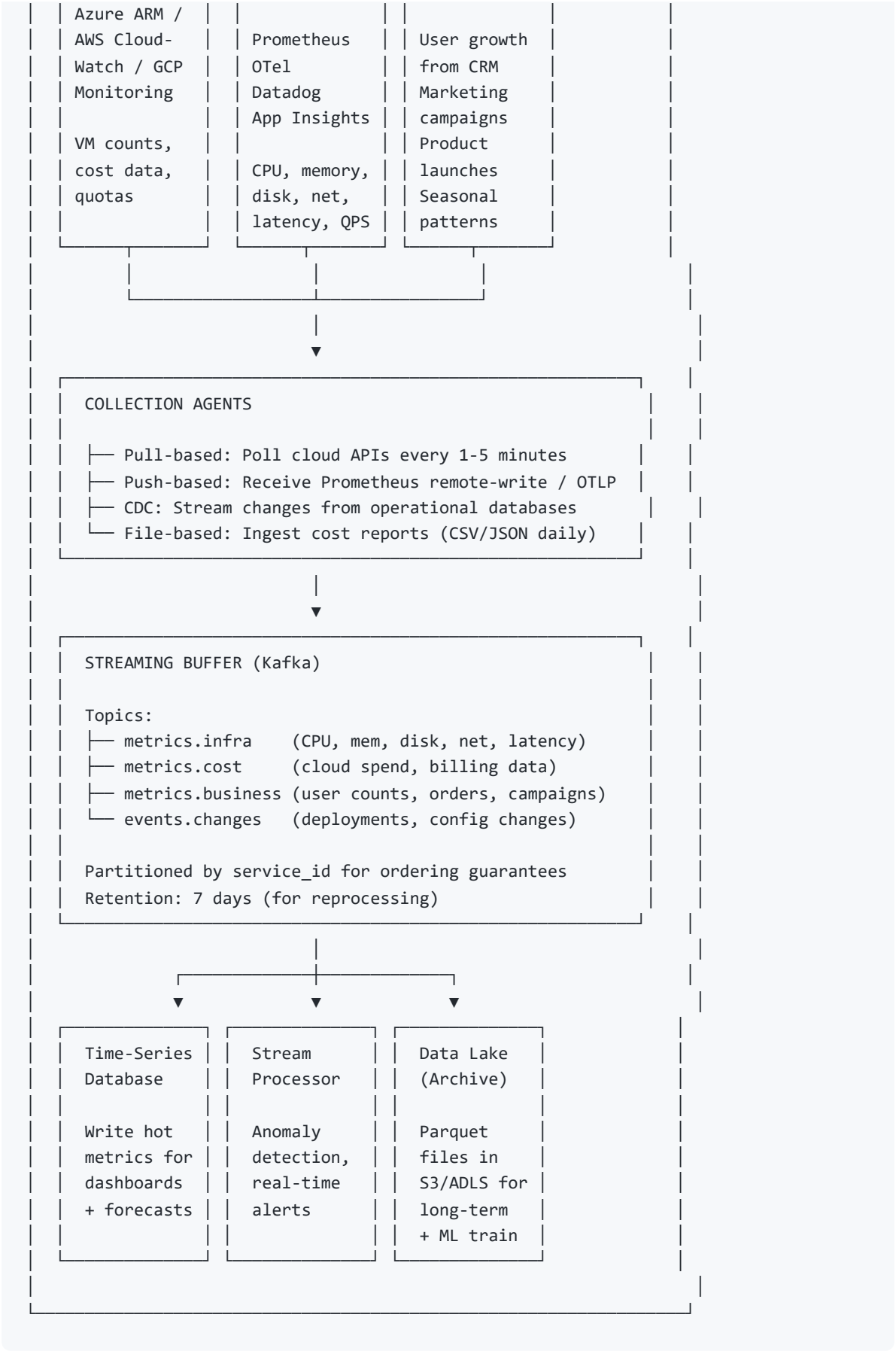
What-If Simulation Request / Response

```
POST /v1/simulate
{
  "scenario": "region_failure",
  "parameters": {
    "failed_region": "eastus",
    "duration_hours": 4,
    "failover_target": "westus2"
  },
  "scope": ["payment-api", "order-service", "user-service"]
}

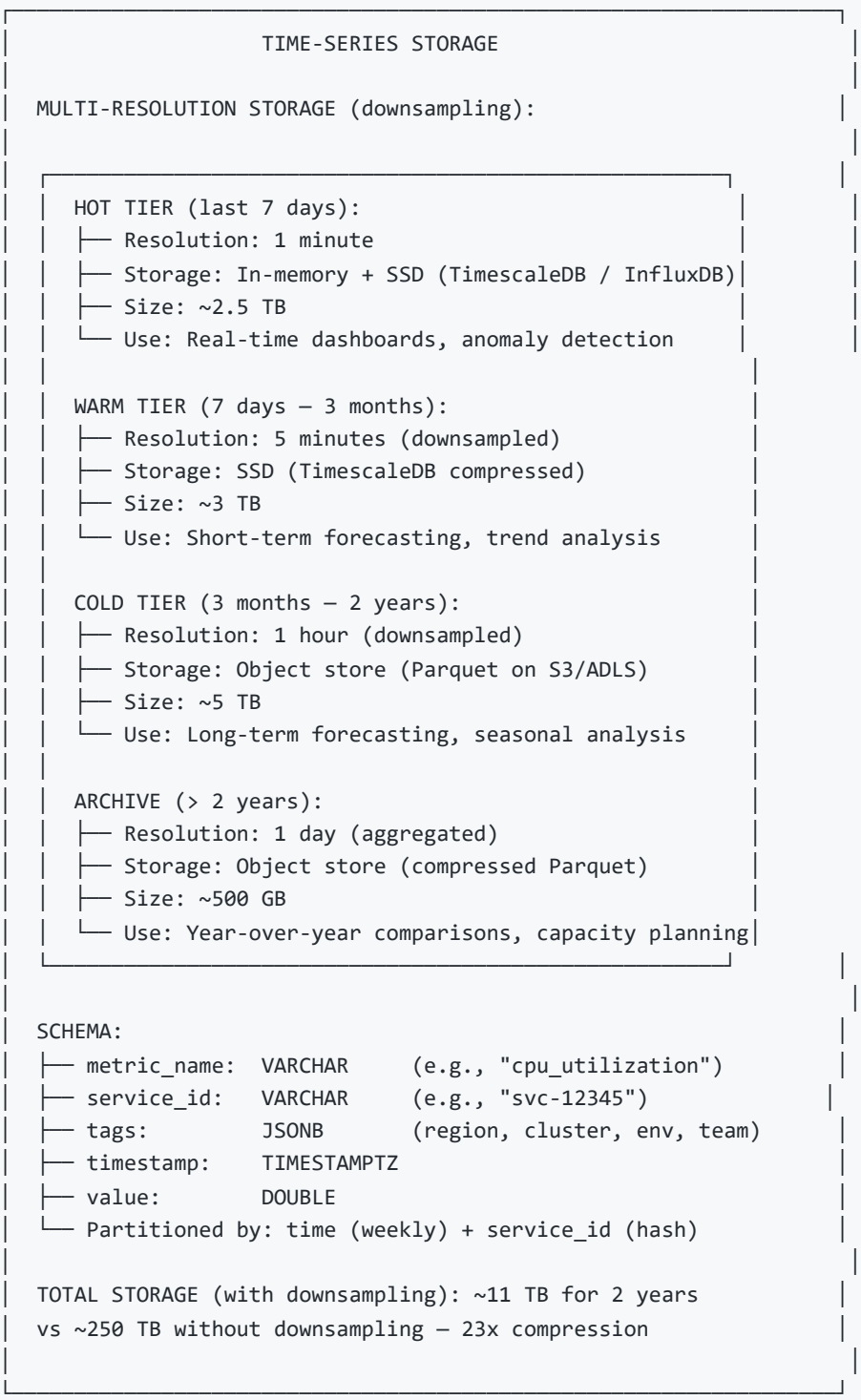
{
  "scenario": "region_failure",
  "summary": "East US failure would require failover of 342 VMs across 3 services to
  "impact": {
    "services_affected": 3,
    "vms_to_failover": 342,
    "current_westus2_headroom_percent": 28,
    "headroom_after_failover_percent": -14,
    "additional_capacity_needed": "85 D8s_v5 VMs",
    "estimated_failover_time_minutes": 12,
    "latency_increase_ms": 45,
    "additional_cost_per_hour": "$2,800"
  },
  "recommendation": "Pre-provision 85 warm standby VMs in West US 2 to handle East U
  "risk_level": "high"
}
```

Data Ingestion Pipeline

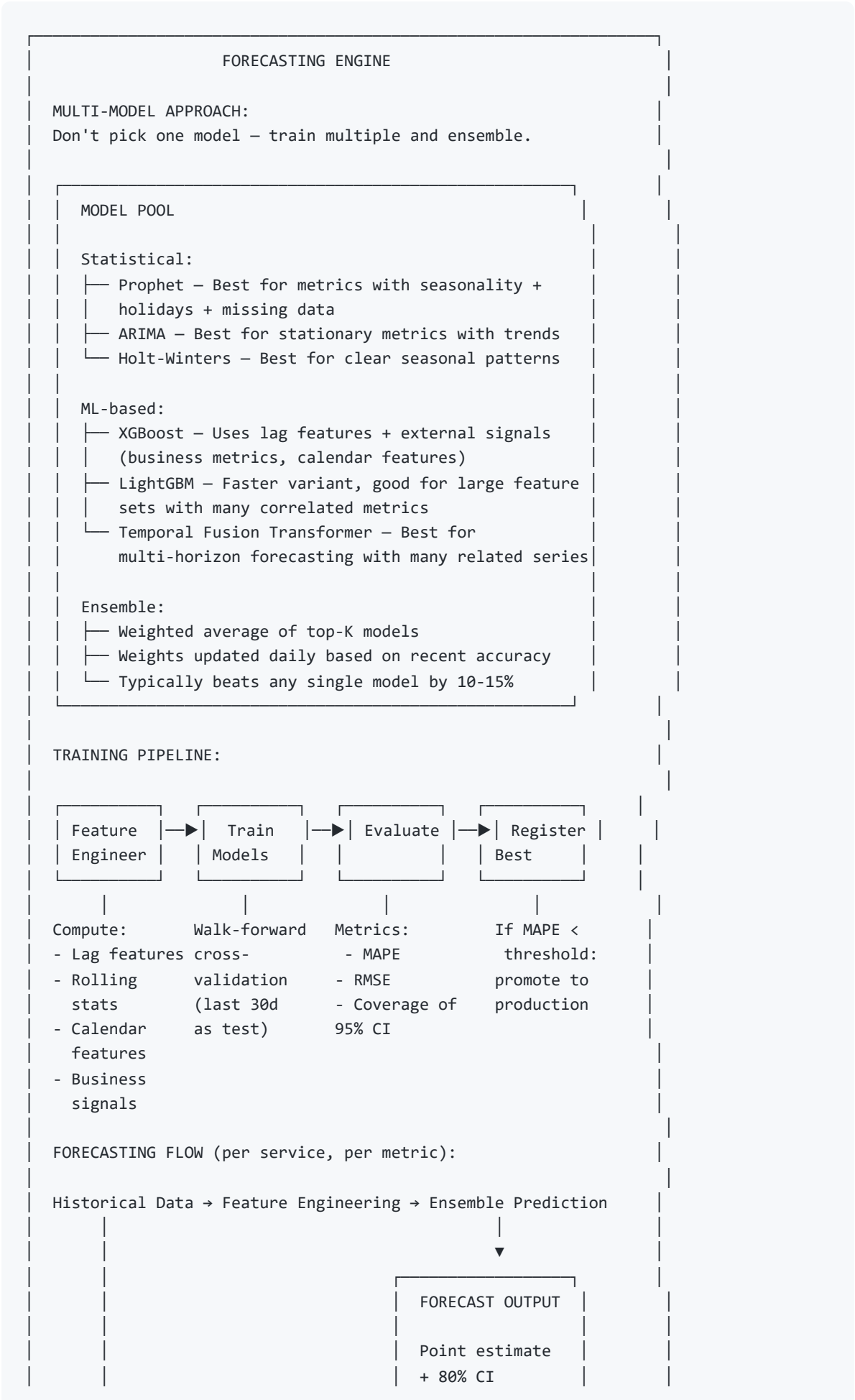


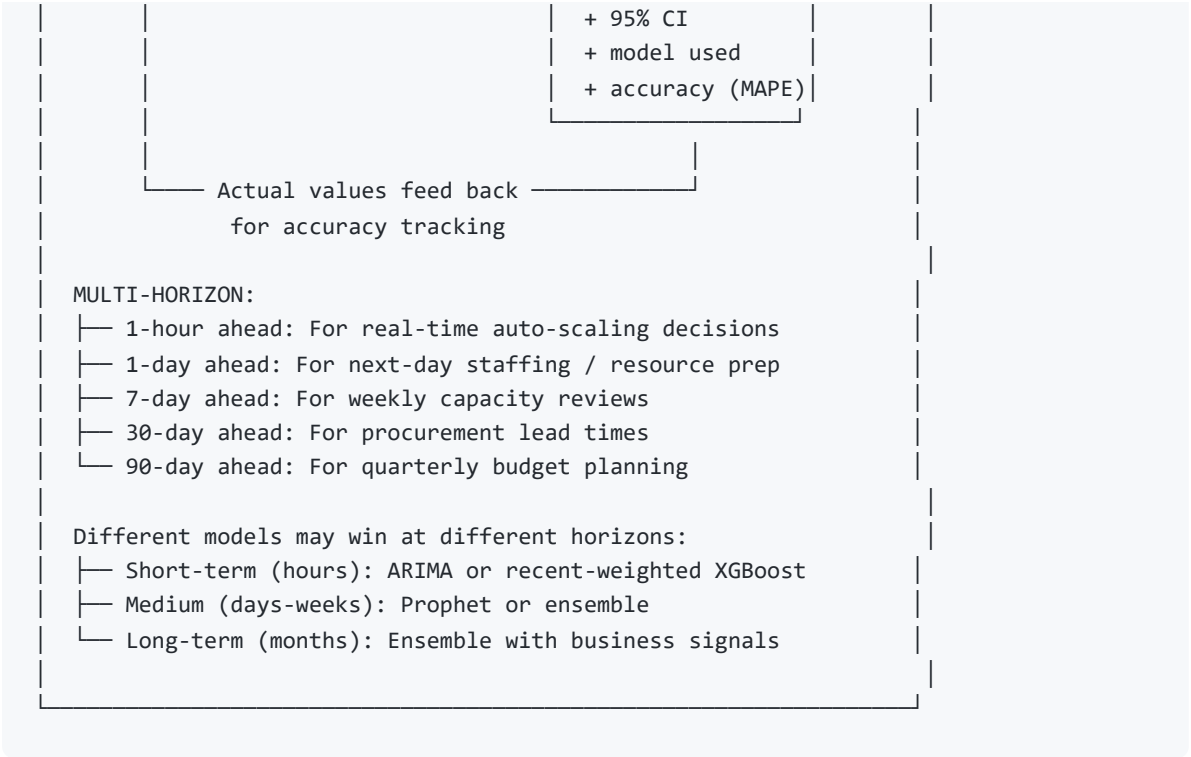

Azure ARM / AWS Cloud-Watch / GCP Monitoring
VM counts, cost data, quotas
Prometheus OTel Datadog App Insights
CPU, memory, disk, net, latency, QPS
User growth from CRM Marketing campaigns Product launches Seasonal patterns
COLLECTION AGENTS
├── Pull-based: Poll cloud APIs every 1-5 minutes
├── Push-based: Receive Prometheus remote-write / OTLP
├── CDC: Stream changes from operational databases
└── File-based: Ingest cost reports (CSV/JSON daily)
STREAMING BUFFER (Kafka)
Topics:
├── metrics.infra (CPU, mem, disk, net, latency)
├── metrics.cost (cloud spend, billing data)
├── metrics.business (user counts, orders, campaigns)
└── events.changes (deployments, config changes)
Partitioned by service_id for ordering guarantees
Retention: 7 days (for reprocessing)
Time-Series Database
Write hot metrics for dashboards + forecasts
Stream Processor
Anomaly detection, real-time alerts
Data Lake (Archive)
Parquet files in S3/ADLS for long-term + ML train

Time-Series Storage

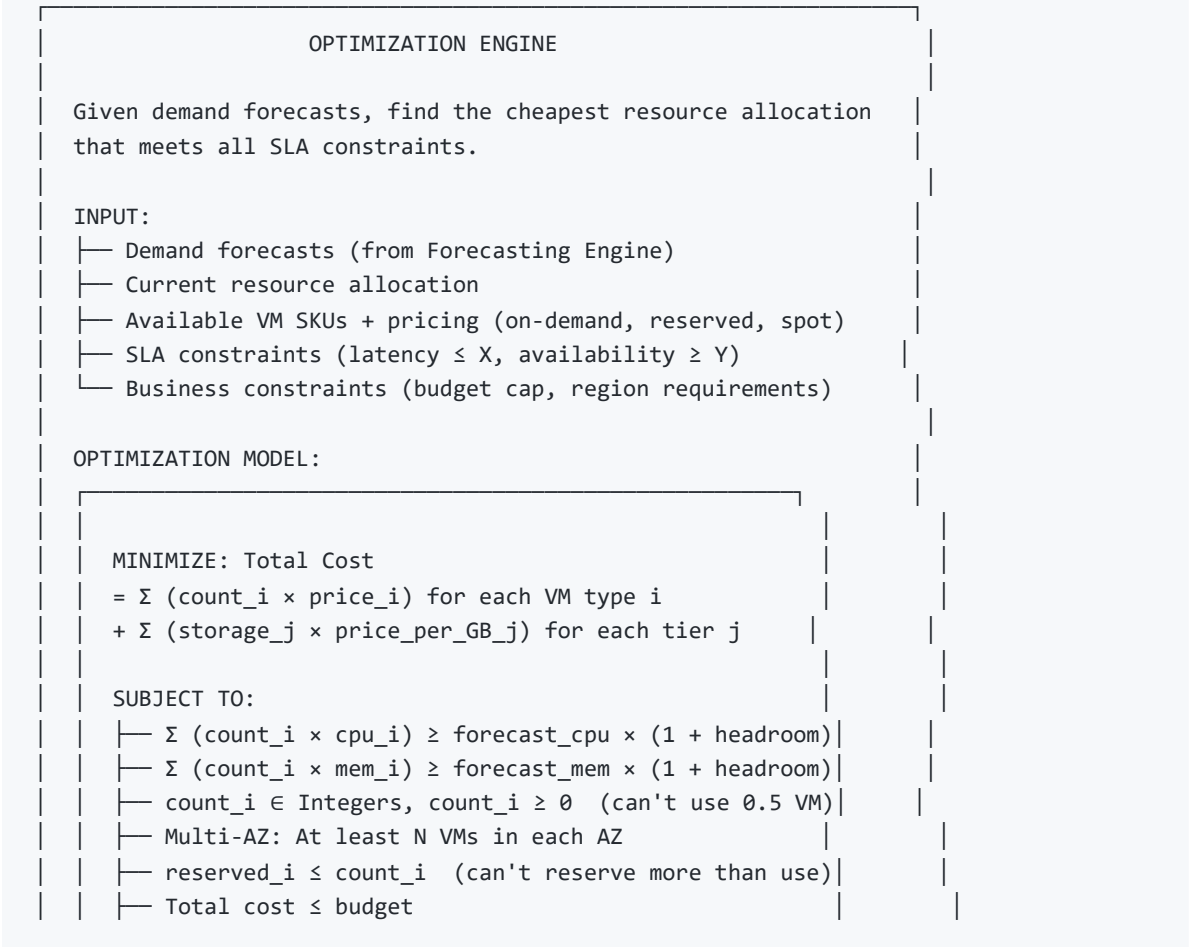


Forecasting Engine


FORECASTING ENGINE
MULTI-MODEL APPROACH:
Don't pick one model — train multiple and ensemble.
MODEL POOL
Statistical:
├── Prophet — Best for metrics with seasonality + holidays + missing data
├── ARIMA — Best for stationary metrics with trends
└── Holt-Winters — Best for clear seasonal patterns
ML-based:
├── XGBoost — Uses lag features + external signals (business metrics, calendar features)
├── LightGBM — Faster variant, good for large feature sets with many correlated metrics
└── Temporal Fusion Transformer — Best for multi-horizon forecasting with many related series
Ensemble:
├── Weighted average of top-K models
├── Weights updated daily based on recent accuracy
└── Typically beats any single model by 10-15%
TRAINING PIPELINE:
Feature Engineer
Train Models
Evaluate
Register Best
Compute:
- Lag features
- Rolling stats
- Calendar features
- Business signals
Walk-forward cross-validation (last 30d as test)
Metrics:
- MAPE
- RMSE
- Coverage of 95% CI
If MAPE < threshold: promote to production
FORECASTING FLOW (per service, per metric):
Historical Data → Feature Engineering → Ensemble Prediction
FORECAST OUTPUT
Point estimate + 80% CI



Optimization Engine



```
│   │   └── Regional constraints (data residency, latency)     │        │
│   │                                                          │        │
│   │   SOLVER: Mixed Integer Programming (OR-Tools, Gurobi)   │        │
│   │   Solve time: < 30 seconds for 10K services              │        │
│   └──────────────────────────────────────────────────────────┘        │
│                                                                       │
│   OUTPUT:                                                             │
│   ┌──────────────────────────────────────────────────────────┐        │
│   │   RECOMMENDATIONS:                                       │        │
│   │                                                          │        │
│   │   Service: payment-api                                   │        │
│   │   ├── Current: 80× D4s_v5 on-demand ($12,800/mo)         │        │
│   │   ├── Recommended: 50× D4s_v5 reserved + 15× D8s_v5      │        │
│   │   │   on-demand ($8,900/mo)                              │        │
│   │   ├── Savings: $3,900/month (30.5%)                      │        │
│   │   ├── Headroom after: 34% CPU, 28% memory                │        │
│   │   ├── Risk: Low (forecast confidence 92%)                │        │
│   │   └── Action: Convert 50 VMs to 1-year RI, resize 15     │        │
│   │                                                          │        │
│   │   Service: ml-training-cluster                           │        │
│   │   ├── Current: 20× NC24s_v3 (GPU) on-demand              │        │
│   │   ├── Recommended: Add 8× NC24s_v3 by April 1            │        │
│   │   ├── Reason: Demand forecast shows 82% GPU utilization  │        │
│   │   ├── Cost increase: $45,000/month                       │        │
│   │   └── Alternative: Use spot instances (save 60%, 5% risk)│        │
│   └──────────────────────────────────────────────────────────┘        │
│                                                                       │
│   RESERVED INSTANCE OPTIMIZATION:                                     │
│   ├── Analyze stable baseline usage per service                       │
│   ├── Recommend RI coverage for baseline (save 30-60%)                │
│   ├── Keep burst capacity as on-demand or spot                        │
│   └── Consider RI flexibility (instance family flexibility)           │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

LLM-Assisted Interface

```
┌──────────────────────────────────────────────────────────────────────┐
│                    LLM-ASSISTED INTERFACE                            │
│                                                                      │
│   The LLM acts as an ORCHESTRATOR — it doesn't do math or            │
│   forecasting itself. It translates natural language to API          │
│   calls, then formats the results for humans.                        │
│                                                                      │
│   ARCHITECTURE (Agent / ReAct Pattern):                              │
│                                                                      │
│   User Question                                                      │
│        │                                                             │
│        ▼                                                             │
│   ┌──────────────────────────────────────────────────────┐           │
│   │   LLM (GPT-4 / Claude)                               │           │
```

```
│  │                                                    │        │
│  │  System Prompt:                                    │        │
│  │  "You are a capacity planning assistant. You have  │        │
│  │   access to these tools: forecast_api, optimize_api,  │     │
│  │   simulate_api, metrics_api, knowledge_search.     │        │
│  │   Always use tools for data — never guess numbers."  │      │
│  │                                                    │        │
│  │  ReAct Loop:                                       │        │
│  │  1. THINK: "User wants GPU forecast for Q2.        │        │
│  │     I need to call forecast_api for GPU metrics."  │        │
│  │  2. ACT: forecast_api(service="ml-cluster",        │        │
│  │          metric="gpu_util", horizon=90)            │        │
│  │  3. OBSERVE: {forecast data with confidence intervals}  │   │
│  │  4. THINK: "I should also check the ML roadmap for │        │
│  │     planned workloads..."                          │        │
│  │  5. ACT: knowledge_search("ML platform Q2 roadmap")  │      │
│  │  6. OBSERVE: {relevant doc chunks}                 │        │
│  │  7. THINK: "Now I have data + context. Generate answer."│   │
│  │  8. RESPOND: Formatted answer with sources         │        │
│  └────────────────────────────────────────────────────┘        │
│                                                                │
│  AVAILABLE TOOLS:                                              │
│  ├── forecast_api — Get demand forecasts for any metric/service  │
│  ├── metrics_api — Query current and historical metrics        │
│  ├── optimize_api — Run cost optimization for a scope          │
│  ├── simulate_api — Run what-if scenarios                      │
│  ├── knowledge_search — RAG search over internal docs          │
│  ├── alert_api — Check active alerts and risk levels           │
│  └── report_api — Generate formatted reports                   │
│                                                                │
│  GROUNDING (preventing hallucination):                         │
│  ├── ALL numbers come from APIs — LLM never invents numbers    │
│  ├── ALL references cite sources (doc name, API call, metric)  │
│  ├── Confidence levels always shown (from forecast API)        │
│  └── "I don't have data for that" if no relevant tool/data     │
│                                                                │
│  REPORT GENERATION:                                            │
│  ├── Weekly capacity report: LLM calls multiple APIs, assembles │
│  │   data into structured report (markdown / PDF)              │
│  ├── Sections: Executive summary, top risks, cost trends,      │
│  │   upcoming capacity needs, optimization opportunities       │
│  └── Saves 2-3 hours of analyst time per report                │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

ML Model Lifecycle

```
┌──────────────────────────────────────────────────────────────┐
│                   MODEL LIFECYCLE MANAGEMENT                 │
│                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐      │
```



Alert & Action System


ALERT & ACTION SYSTEM
ALERT TYPES:
CAPACITY RISK ALERTS (from forecasting engine):
├── "CPU forecast to exceed 85% in 14 days"
├── "Storage will be full in 21 days"
├── "Network bandwidth approaching limit (78%)"
└── Severity based on time-to-saturation:
Critical: < 7 days, Warning: 7-30 days
ANOMALY ALERTS (from stream processor):
├── "Unexpected CPU spike on order-service (3σ)"
├── "Error rate increased 5x in last 15 minutes"
└── "Cost anomaly: spend 40% above forecast today"
BUDGET ALERTS (from cost tracking):
├── "Team X at 80% of monthly budget with 10d left"
├── "Reserved instance utilization below 60%"
└── "Savings opportunity: $30K/mo from right-sizing"
ACTION ROUTING (Risk-Based):
Alert → Classify Risk → Route:
├── LOW RISK → Auto-execute:
│ Scale up by 1-5 pods within HPA limits
│ Restart unhealthy instances
│ Clear cache, compact logs
├── MEDIUM RISK → Execute + Notify:
│ Scale up by 10+ VMs (within pre-approved limits)
│ Switch traffic between AZs
│ Trigger model retraining
└── HIGH RISK → Require Approval:
Add > $10K/month in resources
Change architecture (new regions, new tiers)
Modify production ML model configuration
Decommission resources
NOTIFICATION CHANNELS:
├── PagerDuty / OpsGenie (critical — wake up on-call)
├── Slack / Teams (warning — team channel)
├── Email (informational — weekly digest)
└── Dashboard (all — always visible)

Key Components

Component	Technology	Purpose
Metrics Ingestion	Kafka + OTel Collectors	Stream infrastructure and business metrics
Time-Series DB	TimescaleDB / InfluxDB / Prometheus (Thanos)	Store and query time-series metrics with multi-resolution
Data Lake	S3/ADLS + Delta Lake (Parquet)	Archive historical metrics for ML training
Forecasting Engine	Prophet + XGBoost + Ensemble (Python)	Predict future resource demand
Optimization Engine	Google OR-Tools / Gurobi (MIP solver)	Find cost-optimal resource allocation
Model Registry	MLflow	Track experiments, version models, promote to production
Feature Store	Feast	Consistent features for training and batch inference
LLM Interface	GPT-4 / Claude + ReAct agent framework	Natural language queries, report generation
Knowledge Base	pgvector / Qdrant + embedding model	RAG over internal docs, runbooks, past reports
Dashboard	Grafana + custom React UI	Interactive capacity views, forecast visualization
Alert System	Prometheus Alertmanager + PagerDuty	Risk-based alerting with auto-action routing
Report Generator	LLM + templates + chart library	Automated weekly/monthly capacity reports
Scheduler	Airflow / Prefect	Orchestrate training, forecasting, optimization jobs
Audit Log	Append-only log (Kafka → DB)	Record all recommendations, decisions, actions

Scalability Considerations

1. Metrics Ingestion Scalability

```
Challenge: 5M+ data points per minute, growing with infrastructure

Solutions:
├── Kafka partitioning: Partition by service_id for parallelism
├── OTel collectors: Horizontally scale collectors per region
├── Pre-aggregation: Aggregate at collector level (1s → 1min)
│   before writing to TSDB (reduces write volume 60x)
├── Backpressure: Kafka decouples producers from consumers
│   → spikes buffered, consumers process at their own pace
└── Write batching: Batch 1,000+ data points per TSDB write
```

2. Forecast Computation Scalability

```
Challenge: Forecast 5M time-series daily across multiple horizons

Solutions:
├── Parallel execution: Each service forecasted independently
│   → Embarrassingly parallel across Spark/Dask workers
├── Model caching: Pre-trained models cached, only re-fit weekly
│   → Daily forecast = inference only (fast)
├── Prioritized scheduling: Critical services forecasted first
│   → Long-tail services can be batched in off-peak hours
├── Pre-computed features: Feature store serves pre-computed
│   lag/rolling features → no real-time feature engineering
└── GPU for neural models: Batch inference on GPU for TFT model
    → 10x faster than CPU for neural forecasts
```

3. LLM Service Scalability

```
Challenge: LLM calls are expensive (latency + cost)

Solutions:
├── Caching: Cache LLM responses for identical/similar queries
│   → "What's the CPU forecast for payment-api?" cached for 1hr
├── Semantic cache: Embed query → check if similar query cached
├── Tiered models: Simple queries → smaller/cheaper model
│   Complex analysis → GPT-4 / Claude Sonnet
├── Pre-generated reports: Batch-generate common reports
│   → LLM only called for custom/ad-hoc queries
└── Rate limiting: Max 100 LLM queries/user/day
```

Trade-offs

Decision	Option A	Option B	Platform Choice
Time-series DB	Prometheus + Thanos	TimescaleDB	**TimescaleDB** — SQL interface, better for complex analytical queries + forecasting
Forecasting approach	Single best model	Multi-model ensemble	**Ensemble** — no single model wins always; ensemble improves accuracy 10-15%
Optimization solver	Greedy heuristic	MIP solver (OR-Tools)	**MIP solver** — provably optimal solutions, fast enough for nightly runs
LLM grounding	Fine-tuned model	RAG + base model	**RAG** — easier to update knowledge, no training needed, better auditability
Alert automation	All human-approved	Risk-tiered auto-action	**Risk-tiered** — auto for low-risk, human approval for high-risk
Storage architecture	Single TSDB	Multi-tier (hot/warm/cold)	**Multi-tier** — 23x storage reduction with downsampling, acceptable accuracy loss
Forecast frequency	On-demand only	Batch + on-demand	**Batch + on-demand** — pre-compute daily forecasts, allow on-demand for ad-hoc analysis
Model retraining	Scheduled only	Scheduled + drift-triggered	**Both** — scheduled as baseline, drift-triggered for sudden changes

---

Technologies Deep Dive

Data Layer

Technology	Purpose	Why

TimescaleDB	Time-series storage + queries	SQL-native, compression, continuous aggregates, mature
Kafka	Metrics streaming buffer	High throughput, partitioned, replay for reprocessing
Delta Lake / Parquet	Historical archive on object store	Cheap, columnar, schema evolution, ACID on data lake
pgvector	Vector store for RAG knowledge base	Embedded in PostgreSQL, simple ops, good enough scale

ML/AI Layer

Technology	Purpose	Why
Prophet	Statistical forecasting	Auto-handles seasonality, holidays, missing data
XGBoost / LightGBM	ML-based forecasting	Handles tabular features (business signals, lag features)
MLflow	Experiment tracking + model registry	Open-source, widely adopted, tracks full lineage
Feast	Feature store	Open-source, offline + online serving, prevents training-serving skew
OR-Tools	MIP optimization solver	Free, fast, well-documented, good for resource allocation
GPT-4 / Claude	LLM for NL interface	Best reasoning for complex queries, good tool use

Infrastructure

Technology	Purpose	Why
Airflow / Prefect	Pipeline orchestration	Schedule training, forecasting, optimization jobs
Grafana	Dashboards and visualization	Industry standard, native TSDB integration
PagerDuty	Alert routing and on-call	Best-in-class incident management integration

Kubernetes	Service orchestration	Auto-scaling, rolling deploys for all platform services

---

How to Explain It Well

30-Second Pitch

> "This is an AI-powered capacity planning platform that collects infrastructure and business metrics, forecasts future demand using an ensemble of statistical and ML models, and optimizes resource allocation using a constraint solver to minimize cost while meeting SLAs. The key differentiator is the AI layer: an LLM-powered interface lets engineers ask natural language questions like 'Will we need more GPUs next quarter?' — the LLM orchestrates API calls to the forecasting and simulation engines, grounds the answer in real data and internal docs via RAG, and generates automated reports. Risk-tiered automation handles low-risk actions automatically while requiring human approval for high-risk decisions."

Key Talking Points

1. **Multi-model ensemble**: No single forecasting model wins always — ensemble of Prophet + XGBoost + neural beats any individual model by 10-15%
2. **Optimization is a separate step from forecasting**: Forecasting tells you *how much* you'll need; the MIP solver tells you the *cheapest way* to get it
3. **LLM as orchestrator, not calculator**: The LLM never does math — it translates natural language to structured API calls and formats results for humans
4. **Risk-tiered automation**: Auto-execute low-risk (pod scale), human-approve high-risk (major spend changes). Build trust over time.
5. **Multi-resolution time-series storage**: Hot (1min, 7d) → Warm (5min, 3mo) → Cold (1hr, 2yr) → Archive (1d, 2yr+). 23x storage reduction.

Whiteboard Sequence

1. Start with the data flow: Metrics sources → Kafka → TSDB + Data Lake
2. Draw the forecasting engine: Historical data → multi-model ensemble → forecast + confidence intervals
3. Add the optimization engine: Forecasts + constraints → MIP solver → recommendations
4. Draw the LLM interface: NL query → ReAct agent → API calls → grounded answer
5. Show the feedback loop: Monitor model accuracy → drift detection → auto-retrain

---

Common Follow-up Questions

"Why an ensemble of models instead of just picking the best one?"

- Different models excel at different patterns: Prophet handles seasonality well, XGBoost captures correlations with business signals, ARIMA works best for stationary series
- The "best" model changes over time as data patterns evolve — what was best last month may not be best this month
- Ensemble (weighted average) is robust: even if one model degrades, others compensate
- Weight update mechanism: Recalculate weights daily based on last 7 days of accuracy → automatically adapts
- At Google/Meta, production forecasting systems almost always use ensembles — it's industry best practice

"How do you prevent the LLM from hallucinating numbers?"

- **Architecture enforces grounding**: The LLM has NO direct access to data. It can only call structured APIs (forecast_api, metrics_api, etc.)
- **Tool use pattern**: LLM calls forecast_api → gets JSON with exact numbers → formats into natural language
- **RAG for context**: When referencing internal docs, the LLM uses vector search to retrieve actual document chunks, then quotes them
- **Citation requirement**: System prompt requires the LLM to cite sources for every claim ("Source: forecast_api response, confidence: 87%")
- **Guardrail check**: Post-generation validation — if LLM mentions a number not found in any API response, flag as potential hallucination

"How would you handle a sudden change like COVID that breaks all historical patterns?"

- **Detection**: Drift detection fires immediately — MAPE increases sharply, feature distributions shift dramatically
- **Short-term**: Fall back to simple heuristics (e.g., "keep current capacity + 20% headroom") until models can adapt
- **Medium-term**: Retrain models on post-change data only (e.g., last 2 weeks), accepting lower confidence
- **Long-term**: Add the disruption as a feature (binary flag: "pre-COVID" / "post-COVID"), retrain on full dataset
- **Key insight**: No model predicts black swans. The platform's value is in **detecting the change quickly** and **providing tools for humans to respond** (what-if simulations, manual overrides)

"How do you decide the headroom target (30% vs 40% vs 50%)?"

- It's a **cost-risk trade-off**: more headroom = more cost but less risk of capacity-related outages
- We calibrate using **historical data**: look at the largest demand spike in the past year that wasn't predicted by the model. Set headroom to cover that spike.
- Different services get different headroom: **revenue-critical** (payment-api): 40%. **Internal tool**: 20%. **ML training**: 15% (can tolerate delays).
- Headroom also depends on **scaling speed**: if you can auto-scale in 2 minutes, you need less headroom than if provisioning takes 2 weeks
- The optimization engine treats headroom as a **tunable parameter** — you can ask "what would our spend be at 30% vs 40% headroom?" and compare

"Why TimescaleDB over Prometheus for the TSDB?"

- **Prometheus** is excellent for real-time monitoring and alerting, but:
  - Limited query language (PromQL) — hard to do complex joins, aggregations needed for forecasting
  - Not designed for long-term storage (Thanos/Cortex add complexity)
  - Pull-based model doesn't scale well beyond 10M+ series
- **TimescaleDB** provides:
  - Full SQL — can do complex analytical queries needed for ML feature engineering
  - Built-in compression (95%+) and continuous aggregates (automatic downsampling)
  - Works as both TSDB and regular PostgreSQL — one database for metrics + metadata + forecasts
  - Better suited for the analytical workloads that capacity planning demands
- In practice: Many teams use **both** — Prometheus for real-time alerting, TimescaleDB for analytics/forecasting (with remote-write to bridge them)

"What if the optimization engine recommends changes that conflict with each other?"

- The MIP solver handles this **by design**: all services are optimized simultaneously within global constraints (total budget, available quotas per region)
- Conflicting recommendations can't happen because the solver satisfies all constraints simultaneously
- Example: If service A and service B both need more compute in East US, but the region quota is limited → the solver finds the globally optimal allocation across both services
- For **sequential changes** (e.g., "move traffic before adding capacity"), the optimizer generates an ordered action plan with dependencies

- Human review catches any constraint the model didn't encode — another reason high-risk recommendations require approval